

Genomics

Leukemia is a cancer of blood-generating tissues. Over 475,000 Americans have Leukemia or are in remission from it. It accounts for 3.3% of all new cancer cases and 3.8% of cancer deaths, with an estimated 66,890 new cases and 23,540 deaths in the U.S. in 2025.

There are two major leukemia families: Acute Lymphoblastic Leukemia (ALLB and ALLT, or ALL), which is cancer of immature lymphoid cells, and Acute Myeloid Leukemia (AML), which is cancer of cancer of immature myeloid cells.

Golub et al. (*Science*, 1999) popularized a dataset including about 7000 genes from 72 patients. The goal is to use genomics data to predict which patients are at risk of ALL versus AML, because the distinction is critical for timely and effective treatment.

1. Load the `golub.csv` dataset. Relabel all instances of ALLB and ALLT as 0, and all instances of ALL as 1. This is the target variable.
2. Use Linear Regression of the target variable on all of the genes provided. What is your mean squared error? Make a kernel density plot of your residuals, and a scatter plot comparing predicted and actual outcomes.
3. Use cross validation to compute the mean squared error of the linear model. Discuss your results from the perspective of the bias variance trade-off.
4. Use the cross validated LASSO to select a set of highly predictive genes. Which set of genes is selected? How many genes are discarded from the model? Make a scatterplot of your predictions versus the actual values.
5. Make a plot that shows the cross validated MSE as *alpha* varies. For what values of α is the LASSO underfitting? Overfitting? What is the optimal penalty hyperparameter that minimizes expected MSE?
6. Explain why linear regression performs perfectly on the training set, but the LASSO provides better predictions overall.
7. Why do regularization methods lend themselves to scenarios like precision health?
8. What are the risks of applying methods like the Lasso to precision health questions, where interventions will then be taken to optimize patient health?

```
In [8]: import pandas as pd
df = pd.read_csv('data/golub.csv')

# 1. Print the first few rows and columns
print("---- Data Shape ----")
print(df.shape)
print("\n---- First 5 Column Names ----")
print(df.columns[:5].tolist())
print("\n---- First 2 Rows ----")
print(df.head(2))
```

```
---- Data Shape ----
(72, 7135)
```

```
---- First 5 Column Names ----
['Samples', 'BM.PB', 'Gender', 'Source', 'tissue.mf']
```

```
---- First 2 Rows ----
   Samples BM.PB Gender Source tissue.mf cancer AFX-BioB-5_at \
0        39    BM     F   DFCI      BM:f   allB   -1363.276427
1        40    BM     F   DFCI      BM:f   allB   -796.285053

   AFX-BioB-M_at AFX-BioB-3_at AFX-BioC-5_at ... U48730_at \
0   -1058.585495   -541.469194    74.349803 ...   -35.081447
1   -1167.103365    7.538493    83.544731 ...  -404.737767

   U58516_at   U73738_at   X06956_at   X16699_at   X83863_at   Z172
40_at \
0  1565.618803 -485.680714 -170.261227 -919.114293  1675.050053  389.7
69289
1   622.498054 -1275.354673  214.828233 -750.220666 -441.589276   9.8
41713

   L49218_f_at M71243_f_at Z78285_f_at
0   -526.449219 -268.963924 -779.643092
1   -671.911209 -671.911209 -644.272577
```

```
[2 rows x 7135 columns]
```

```
In [9]: #1
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression, LassoCV
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import cross_val_score, KFold

df = pd.read_csv('data/golub.csv')
```

```
mapping = {'allB': 0, 'allT': 0, 'all': 1, 'aml': 1}
y = df['cancer'].map(mapping).values

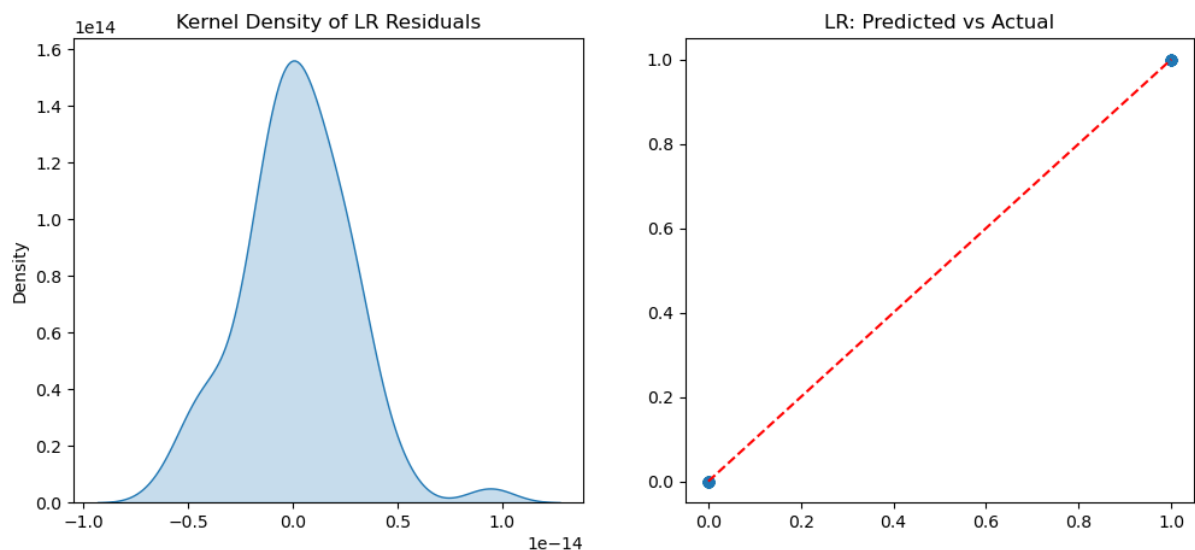
# Identify gene columns (dropping metadata)
X = df.drop(columns=['Samples', 'BM.PB', 'Gender', 'Source', 'tissue.m
```

```
In [ ]: #2
lr = LinearRegression()
lr.fit(X, y)
y_pred_lr = lr.predict(X)
mse_lr = mean_squared_error(y, y_pred_lr)
residuals = y - y_pred_lr

print(f"2. Linear Regression Training MSE: {mse_lr:.4e}")

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
sns.kdeplot(residuals, fill=True, ax=ax[0])
ax[0].set_title("Kernel Density of LR Residuals")
ax[1].scatter(y, y_pred_lr, alpha=0.5)
ax[1].plot([0, 1], [0, 1], 'r--')
ax[1].set_title("LR: Predicted vs Actual")
plt.show()
```

2. Linear Regression Training MSE: 6.7654e-30



```
In [11]: #3
kf = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(lr, X, y, cv=kf, scoring='neg_mean_squared
print(f"3. Linear Regression CV MSE: {-cv_scores.mean():.4f}")
```

3. Linear Regression CV MSE: 0.0408

3

The training MSE being super close to zero shows that there is low bias and low

variance. The MSE is also very similar to the training error which means the linear relationships in the dataset are consistent.

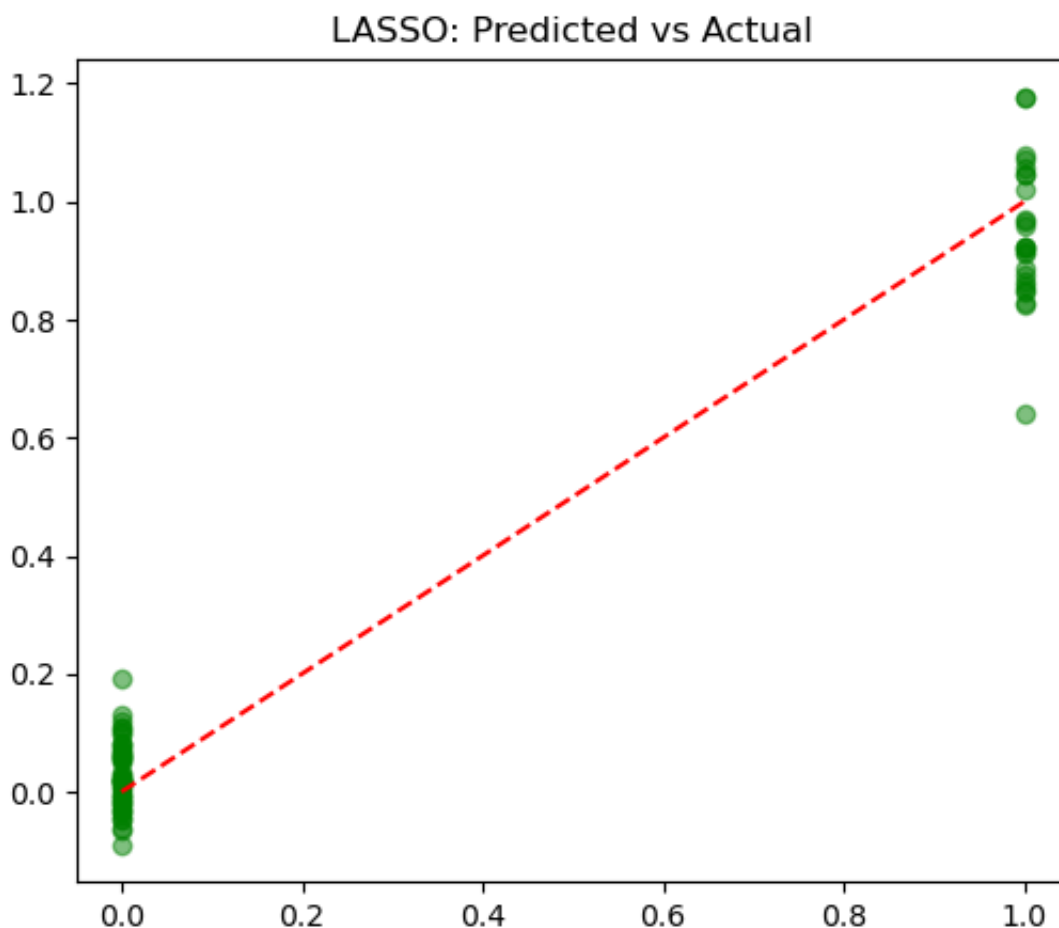
```
In [13]: #4
lasso = LassoCV(cv=5, random_state=30, max_iter=10000)
lasso.fit(X, y)
y_pred_lasso = lasso.predict(X)

selected_genes = X.columns[lasso.coef_ != 0].tolist()
discarded_count = np.sum(lasso.coef_ == 0)

print(f"4. LASSO Selected {len(selected_genes)} genes.")
print(f"    Discarded {discarded_count} genes.")

# Scatterplot for LASSO
plt.figure(figsize=(6, 5))
plt.scatter(y, y_pred_lasso, alpha=0.5, color='green')
plt.plot([0, 1], [0, 1], 'r--')
plt.title("LASSO: Predicted vs Actual")
plt.show()
```

4. LASSO Selected 34 genes.
Discarded 7095 genes.



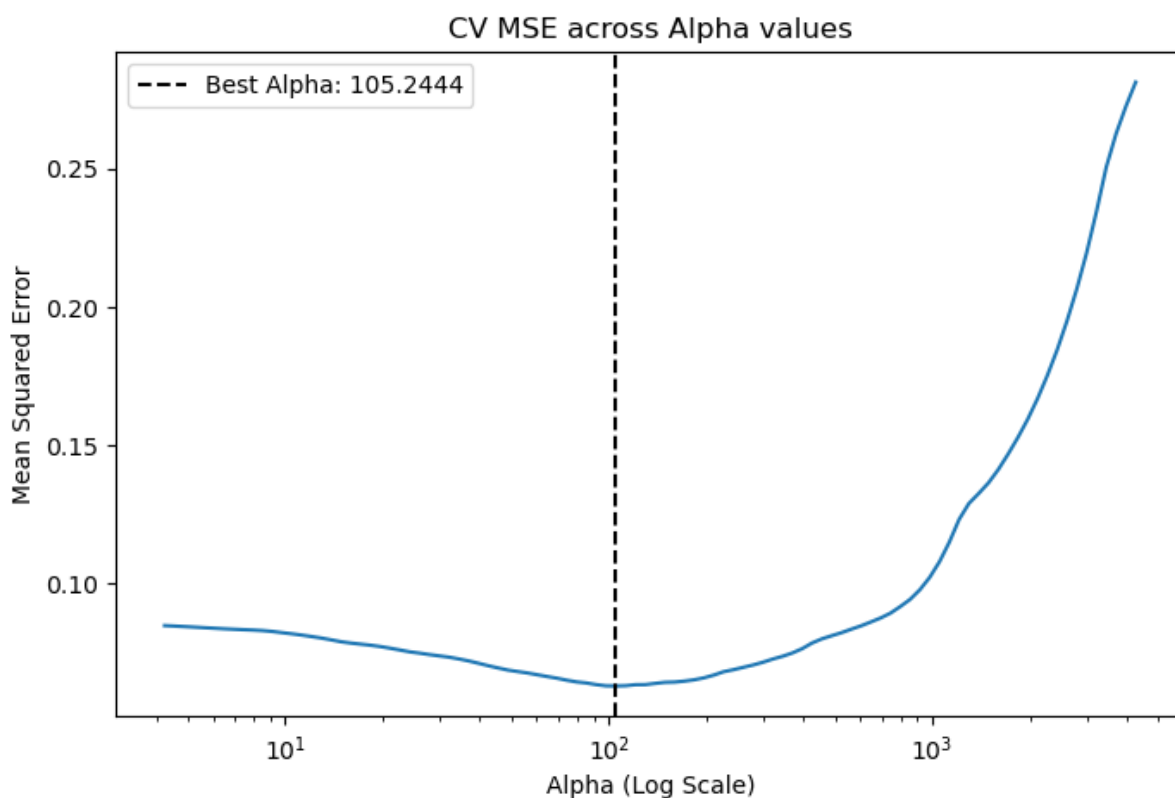
```
In [14]: #5
```

```

mean_mse = lasso.mse_path_.mean(axis=1)
plt.figure(figsize=(8, 5))
plt.semilogx(lasso.alphas_, mean_mse)
plt.axvline(lasso.alpha_, linestyle='--', color='black', label=f'Best
plt.xlabel('Alpha (Log Scale)')
plt.ylabel('Mean Squared Error')
plt.title('CV MSE across Alpha values')
plt.legend()
plt.show()

print(f"5. Optimal Penalty (Alpha): {lasso.alpha_:.6f}")

```



5. Optimal Penalty (Alpha): 105.244367

6

Linear regression is like the LASSO model with no penalty. Because it has 7,000 genes to choose from for only 72 patients, it is able to take advantage of that by memorizing every tiny detail of the training data. This makes it look perfect on paper, but it's actually just fitting noise, which leads to higher MSE when tested on new data.

LASSO is better to use because it applies a penalty that ignores the 7000 noisy variables and focuses only on the few genes that actually matter. This leads to much cleaner predictions.

7

The alpha plot show that error decreases when the coefficients are penalized. In a medical reality, it is not realistic to test every patient for 7,135 genes.

Regularization is able to make sure that the diagnostic model is stable. Without it, adding just one or two new patients to the dataset would completely change which genes the model thinks are important.

8

Underfitting is a risk(look at right side of alpha plot). If the penalty is too high, LASSO might get rid of the gene that could be a vital for curing because it was statistically redundant. If two genes are very correlated, LASSO will choose one and get rid of the other. Once again, this could be getting rid of a key drug.